

TP-1 : Setup Environnement de Développement

Installation Docker et Configuration

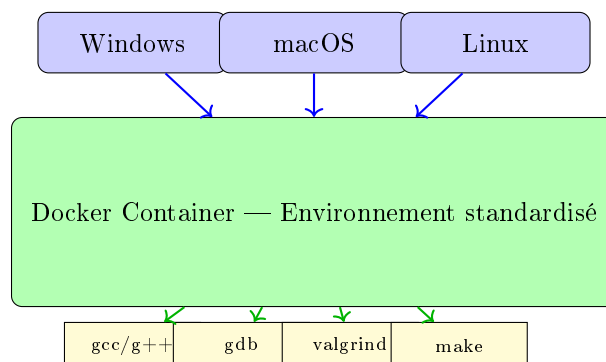
Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

Introduction

Ce TP couvre l'installation et la configuration de l'environnement Docker standardisé pour le cours de Programmation C Avancée. Tous les TP du semestre utiliseront cet environnement pour garantir la cohérence et la reproductibilité.

Pourquoi Docker ?

Docker garantit que tous les étudiants travaillent dans un environnement identique, indépendamment de leur système d'exploitation (Windows, macOS, Linux). Cela évite les problèmes de "ça marche sur ma machine" et facilite le support technique.



Objectifs

À la fin de ce TP, vous serez capable de:

- Installer Docker sur votre système d'exploitation
- Télécharger et exécuter l'image Docker du cours
- **Utiliser Vim pour créer et éditer des fichiers C**
- Compiler et exécuter un programme C simple
- Utiliser les outils de base (gcc, gdb, make)
- Partager des fichiers entre votre système et le container

Prérequis essentiel

La maîtrise de Vim est un prérequis pour tous les TP suivants. Prenez le temps de faire tous les exercices Vim de ce TP. Vous utiliserez Vim dans chaque session de travail du cours.

1 Prérequis Système

Configuration minimale

Matériel:

- Processeur: x86_64 (Intel/AMD) ou ARM64 (Apple Silicon)
- RAM: 4 GB minimum, 8 GB recommandé
- Espace disque: 10 GB disponibles

Systèmes d'exploitation supportés:

- Windows 10/11 (64-bit, avec WSL2)
- macOS 10.15 (Catalina) ou plus récent
- Linux: Ubuntu 20.04+, Debian 10+, Fedora 32+, etc.

2 Installation de Docker

2.1 Windows

Installation Docker Desktop sur Windows

Étape 1: Activer WSL2

Ouvrir PowerShell en mode administrateur et exécuter:

```
wsl --install
```

Redémarrer le système après l'installation.

Étape 2: Télécharger Docker Desktop

Aller sur <https://www.docker.com/products/docker-desktop/>
Télécharger "Docker Desktop for Windows".

Étape 3: Installer

Exécuter l'installateur et suivre les instructions.

Cocher l'option "Use WSL 2 instead of Hyper-V".

Redémarrer après l'installation.

Étape 4: Vérifier

Ouvrir un terminal (Git Bash recommandé, ou PowerShell) et taper:

```
docker --version  
docker run hello-world
```

Si vous voyez le message de bienvenue de Docker, l'installation a réussi.

Problèmes courants Windows

Erreur "WSL 2 installation is incomplete":

```
wsl --update  
wsl --set-default-version 2
```

Erreur de virtualisation: Activer la virtualisation dans le BIOS (VT-x pour Intel, AMD-V pour AMD).

Docker Desktop ne démarre pas: Vérifier que les services WSL sont actifs dans les services Windows.

2.2 macOS

Installation Docker Desktop sur macOS

Étape 1: Télécharger

Aller sur <https://www.docker.com/products/docker-desktop/>
Choisir la version appropriée:

- Mac avec puce Intel: "Docker Desktop for Mac (Intel)"
- Mac avec Apple Silicon (M1/M2/M3): "Docker Desktop for Mac (Apple Silicon)"

Étape 2: Installer

Ouvrir le fichier .dmg téléchargé.
Glisser Docker.app vers Applications.
Lancer Docker depuis Applications.

Étape 3: Vérifier

Ouvrir le Terminal et taper:

```
docker --version
docker run hello-world
```

2.3 Linux

Installation Docker sur Linux (Ubuntu/Debian)

Étape 1: Nettoyer les anciennes versions

```
sudo apt-get remove docker docker-engine docker.io containerd runc
```

Étape 2: Installer les dépendances

```
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg lsb-release
```

Étape 3: Ajouter le dépôt Docker

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Étape 4: Installer Docker

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io \
  docker-compose-plugin
```

Étape 5: Configurer les permissions

```
sudo usermod -aG docker $USER
newgrp docker
```

Étape 6: Vérifier

```
docker --version
docker run hello-world
```

3 Téléchargement de l'Image du Cours

Une fois Docker installé, télécharger l'image du cours:

Image Docker du cours

Nom de l'image: elbachirtoure/c-advanced-course

Versions disponibles:

- v2.0 – Version recommandée (multi-architecture: Intel/AMD et Apple Silicon)

- v1.0 – Version héritée (Intel/AMD uniquement)

Taille: Environ 800 MB

Contenu: gcc, g++, gdb, valgrind, make, cmake, vim, nano

Télécharger l'image

Dans votre terminal, exécuter:

```
docker pull elbachirtoure/c-advanced-course:v2.0
```

Le téléchargement peut prendre quelques minutes selon votre connexion.

Pour vérifier que l'image est présente:

```
docker images
```

Vous devriez voir l'image elbachirtoure/c-advanced-course avec le tag v2.0.

4 Utilisation de Base

4.1 Démarrer un Container

Commandes de lancement

Linux/macOS:

```
docker run -it -v $(pwd):/workspace elbachirtoure/c-advanced-course:v2.0
```

Windows (Git Bash – RECOMMANDÉ):

```
docker run -it -v /${PWD}:/workspace elbachirtoure/c-advanced-course:v2.0
```

Windows (PowerShell):

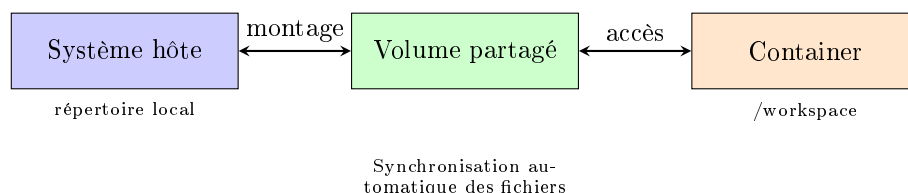
```
docker run -it -v ${PWD}:/workspace elbachirtoure/c-advanced-course:v2.0
```

Explication des options:

- -it: Mode interactif avec terminal
- -v: Monte un volume (partage de fichiers)
- \$(pwd):/workspace: Partage le répertoire courant

Recommandation Windows

Pour les utilisateurs Windows, **Git Bash est fortement recommandé** au lieu de PowerShell. Git Bash offre une expérience Unix-like et évite les problèmes de syntaxe avec le montage de volumes Docker. Git Bash est installé automatiquement avec Git for Windows (<https://git-scm.com/download/win>).



4.2 Navigation dans le Container

Une fois dans le container, vous êtes dans un système Linux minimal.

Commandes de base Linux

```
# Vérifier le répertoire courant (vous êtes déjà dans /workspace)
pwd
```

```
# Lister les fichiers
ls -la

# Créer un répertoire
mkdir mon_projet

# Quitter le container
exit
```

5 Prise en Main de Vim

Vim est obligatoire dans ce cours

Vim est l'éditeur OBLIGATOIRE pour tout le cours de Programmation C Avancée.

Tous les TP supposent que vous utilisez Vim. Cette contrainte n'est pas arbitraire :

- Vim est présent sur tous les systèmes Unix/Linux (serveurs, systèmes embarqués)
- Compétence essentielle pour un ingénieur système
- Productivité maximale une fois maîtrisé
- Préparation aux situations professionnelles sans interface graphique

vs. code (ou nano) est disponible pour débogage uniquement. Toutes les instructions de TP utilisent Vim.

Dans le container Docker, **Vim** est l'éditeur de texte à maîtriser. Cette section vous guide à travers les opérations essentielles d'un programmeur C.

Vim en 3 concepts

1. **Modes:** Vim a plusieurs modes. Le plus important: **ESC** revient toujours au mode NORMAL.
2. **Mode INSERT:** Pour taper du texte, appuyer sur **i**. Pour sortir, appuyer sur **ESC**.
3. **Commandes:** En mode NORMAL, taper **:wq** puis **ENTER** pour sauvegarder et quitter.

Exercice 1: Premiers pas avec Vim

Objectif: Créer et éditer un fichier C avec les opérations de base.

Étape 1: Dans le container, créer un fichier:

```
vim hello.c
```

Vim s'ouvre en mode NORMAL (vous ne pouvez pas taper de texte).

Étape 2: Passer en mode INSERT pour taper du code:

- Appuyer sur **i**
- En bas, vous voyez - INSERT -
- Taper exactement ce code:

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World\n");
5     return 0;
6 }
```

Étape 3: Sauvegarder et quitter:

- Appuyer sur **ESC** (revenir au mode NORMAL)
- Taper **:wq** puis **ENTER**

Étape 4: Vérifier que le fichier existe:

```
cat hello.c
gcc hello.c -o hello
./hello
```

Si vous voyez "Hello World", bravo ! Vous avez créé votre premier fichier avec Vim.

Exercice 2: Édition et navigation

Objectif: Modifier un fichier existant et naviguer dans le code.

Étape 1: Rouvrir le fichier:

```
vim hello.c
```

Étape 2: Navigation (en mode NORMAL):

- j pour descendre, k pour monter
- h pour gauche, l pour droite
- gg pour aller au début du fichier
- G pour aller à la fin

Placer le curseur sur la ligne `printf`.

Étape 3: Ajouter une nouvelle ligne:

- En mode NORMAL, appuyer sur o (ouvre une ligne EN DESSOUS)
- Vim passe automatiquement en mode INSERT
- Taper: `printf("Vim works!\n");`
- Appuyer sur ESC

Étape 4: Copier-coller une ligne:

- Placer le curseur sur une ligne `printf`
- En mode NORMAL, taper yy (copier la ligne)
- Taper p (coller en dessous)

Étape 5: Supprimer une ligne:

- Placer le curseur sur la ligne à supprimer
- Taper dd

Étape 6: Annuler et refaire:

- u pour annuler (undo)
- Ctrl+r pour refaire (redo)

Étape 7: Corriger l'indentation:

- Taper gg pour aller au début
- Taper =G pour auto-indenter tout le fichier

Étape 8: Sauvegarder et tester:

- :wq
- Compiler: `gcc hello.c -o hello`
- Exécuter: `./hello`

Exercice 3: Recherche et remplacement

Objectif: Rechercher et modifier du texte rapidement.

Étape 1: Créer un fichier avec du code à modifier:

vim calcul.c

En mode INSERT (i), taper:

```
1 #include <stdio.h>
2
3 int main() {
4     int x = 10;
5     int y = 10;
6     int z = x + y;
7     printf("Result: %d\n", z);
8     return 0;
9 }
```

Sauvegarder: ESC puis :wq

Étape 2: Rouvrir et rechercher:

vim calcul.c

- Taper /int puis ENTER (cherche "int")
- Taper n pour aller à l'occurrence suivante
- Taper N pour revenir à la précédente

Étape 3: Remplacer toutes les occurrences de "10" par "20":

- Taper :%s/10/20/g puis ENTER
- Toutes les valeurs 10 deviennent 20

Étape 4: Sauvegarder et tester:

- :wq
- gcc calcul.c -o calcul && ./calcul

Sortir de Vim si vous êtes bloqué

Si vous ne savez plus quoi faire dans Vim:

1. Appuyez sur ESC plusieurs fois (revenir au mode NORMAL)
2. Tapez :q! puis ENTER (quitter sans sauvegarder)
3. Ou tapez :wq puis ENTER (sauvegarder et quitter)

5.1 Tableau Récapitulatif des Commandes Vim

Catégorie	Commande	Action
Modes		
Passer en INSERT	i a o O	Insérer avant le curseur Insérer après le curseur Nouvelle ligne en dessous Nouvelle ligne au dessus
Retour NORMAL	ESC	Toujours revenir au mode NORMAL
Navigation		
Déplacement	h j k l w b	Gauche, Bas, Haut, Droite Mot suivant Mot précédent
Lignes	gg G :42	Début du fichier Fin du fichier Aller à la ligne 42
Page	Ctrl+f Ctrl+b	Page suivante Page précédente
Édition		
Supprimer	x dd dw	Supprimer caractère Supprimer ligne Supprimer mot
Copier/Coller	yy p P	Copier ligne Coller après Coller avant
Annuler	u Ctrl+r	Annuler (undo) Refaire (redo)
Recherche		
Chercher	/motif ?motif n N	Chercher vers le bas Chercher vers le haut Occurrence suivante Occurrence précédente
Remplacer	:s/old/new/ :%s/old/new/g	Remplacer sur la ligne Remplacer dans tout le fichier
Sauvegarder/Quitter		
Fichier	:w :q :wq ou :x :q!	Sauvegarder Quitter Sauvegarder et quitter Quitter sans sauvegarder
Spécial C		
Indentation	» «	Indenter ligne Dés-indenter ligne
Navigation code	gg=G %	Auto-indenter tout le fichier Aller à l'accolade/parenthèse correspondante

Pour aller plus loin

Si vous voulez approfondir Vim, le container inclut le tutoriel interactif:

vimtutor

Ce tutoriel de 25-30 minutes couvre toutes les bases de manière interactive.

6 Premier Programme C

Exercice 1: Hello World

Objectif: Compiler et exécuter un programme C dans le container.

Étape 1: Sur votre système (pas dans Docker), créer un répertoire pour le TP:

```
mkdir tp_setup
cd tp_setup
```

Étape 2: Créer un fichier `hello.c` avec votre éditeur préféré:

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello from Docker container!\n");
5     printf("Compilation successful!\n");
6     return 0;
7 }
```

Étape 3: Lancer le container Docker depuis le répertoire `tp_setup`.

Étape 4: Dans le container:

```
gcc -Wall -Wextra -o hello hello.c
./hello
```

Résultat attendu:

```
Hello from Docker container!
Compilation successful!
```

Exercice 2: Vérification des outils

Objectif: Tester tous les outils disponibles dans le container.

2.1) Compiler avec warnings stricts:

```
gcc -Wall -Wextra -pedantic -std=c11 hello.c -o hello
```

2.2) Vérifier la version des outils:

```
gcc --version
gdb --version
make --version
valgrind --version
```

2.3) Tester GDB:

```
gcc -g hello.c -o hello_debug
gdb ./hello_debug
(gdb) break main
(gdb) run
(gdb) next
(gdb) print "GDB works!"
(gdb) quit
```

2.4) Tester Valgrind:

```
valgrind ./hello
```

Vous devriez voir un rapport sans erreurs mémoire.

7 Création d'un Makefile

Exercice 3: Makefile de base

Objectif: Automatiser la compilation avec Make.

Créer un fichier `Makefile` (attention à la casse):

```
1 # Variables
2 CC = gcc
```

```

3 CFLAGS = -Wall -Wextra -std=c11 -g
4 LDFLAGS =
5
6 # Cibles
7 TARGETS = hello
8
9 # Regle par défaut
10 all: $(TARGETS)
11
12 # Compilation de hello
13 hello: hello.c
14     $(CC) $(CFLAGS) -o hello hello.c $(LDFLAGS)
15
16 # Nettoyage
17 clean:
18     rm -f $(TARGETS) *.o
19
20 # Recompilation complete
21 rebuild: clean all
22
23 # Marquer les cibles comme phoniques
24 .PHONY: all clean rebuild

```

Utilisation:

```

make          # Compile
make clean    # Nettoie
make rebuild  # Recompile tout

```

Test: Modifier hello.c, puis taper simplement make.

8 Validation de l'Installation

Exercice 4: Test complet

Objectif: Valider que tout fonctionne correctement.

Créer un programme test_env.c:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     printf("=== Test Environnement ===\n");
6
7     // Test 1: Compilation
8     printf("[OK] Compilation reussie\n");
9
10    // Test 2: Allocation memoire
11    int *ptr = malloc(10 * sizeof(int));
12    if (ptr != NULL) {
13        printf("[OK] malloc fonctionne\n");
14        free(ptr);
15    }
16
17    // Test 3: Arithmetique
18    int result = 42 + 27;
19    if (result == 69) {
20        printf("[OK] Arithmetique correcte\n");
21    }
22
23    printf("\nEnvironnement pret pour le cours!\n");
24    return 0;
25 }

```

Compiler et exécuter:

```

gcc -Wall -Wextra -g test_env.c -o test_env
./test_env
valgrind --leak-check=full ./test_env

```

Résultat attendu:

- Tous les tests [OK]
- Valgrind ne rapporte aucune fuite mémoire

Si tous les tests passent, votre environnement est correctement configuré!

Conclusion

Vous avez maintenant un environnement de développement C professionnel et standardisé. Cet environnement sera utilisé pour tous les TP du cours.

Points clés à retenir

- Toujours lancer Docker depuis le répertoire de travail
- Éditer les fichiers sur votre système, compiler dans Docker
- Utiliser `make` pour automatiser la compilation
- GDB et Valgrind sont vos amis pour le debugging